

ML Part VI Convolutional Neural Networks (CNN)

contribute to Hung-yi Lee

Convolution Neural Networks (CNN)

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

1	0	1
0	1	0
1	0	1

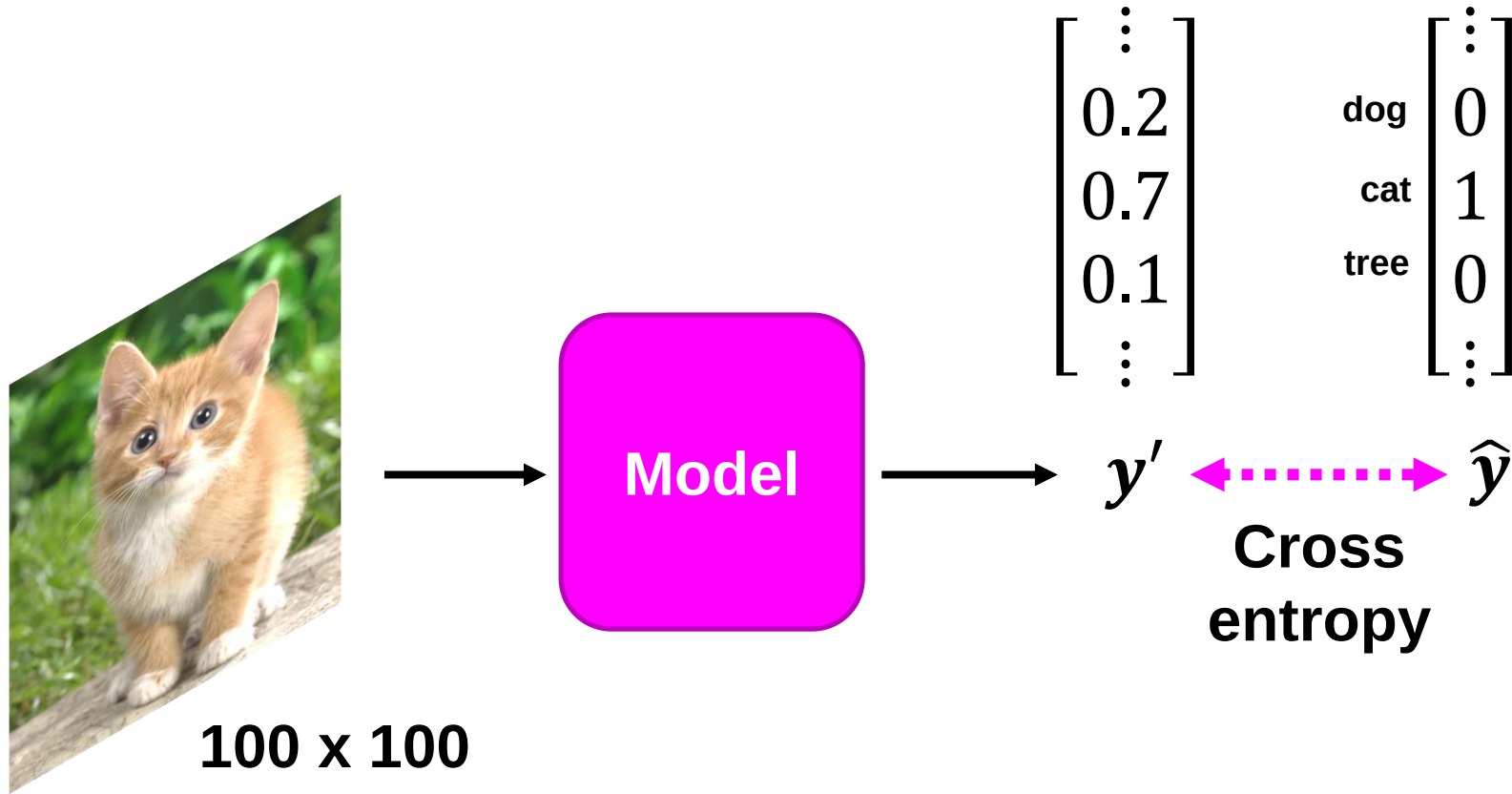
1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

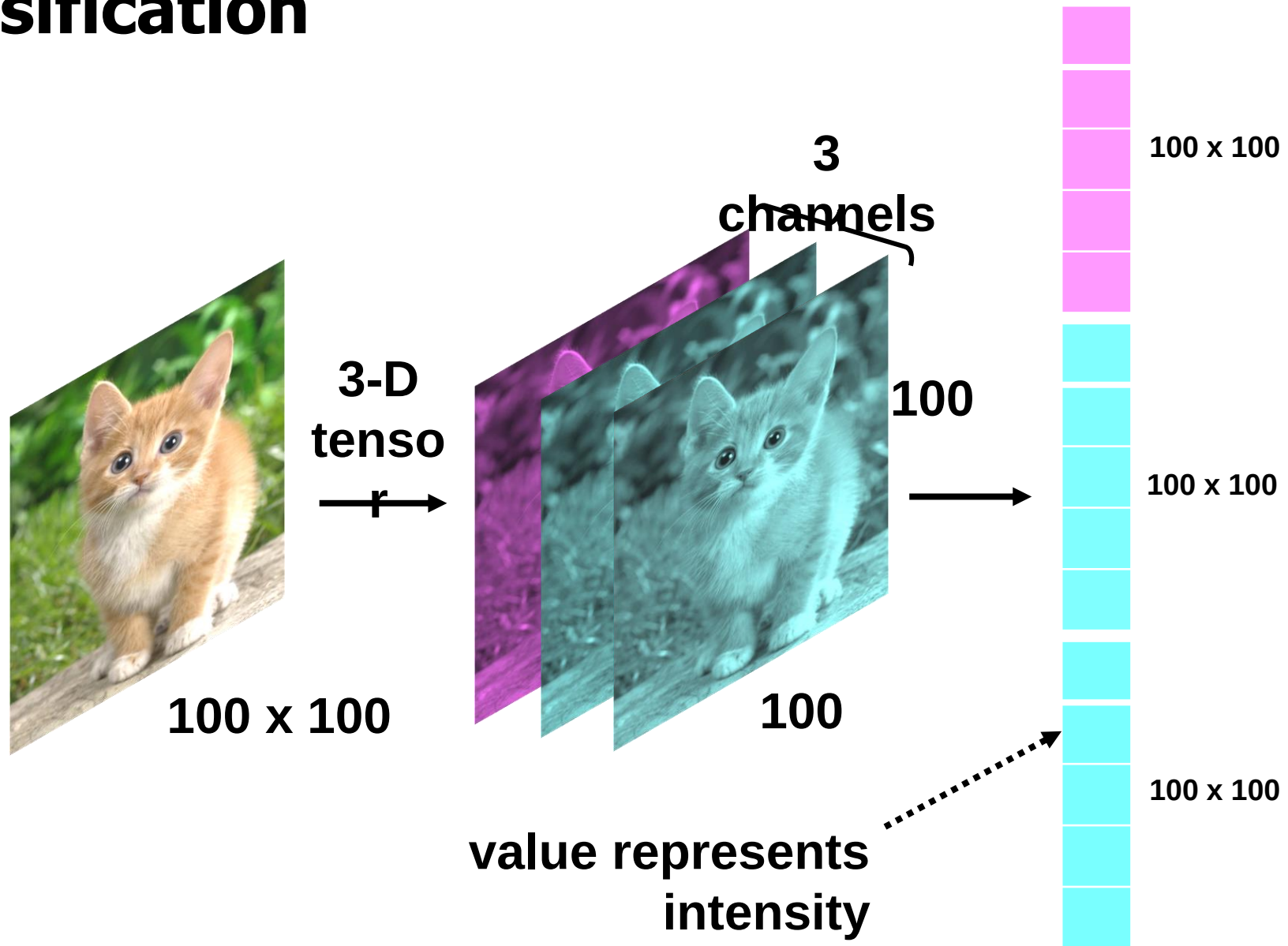
Convolved
Feature

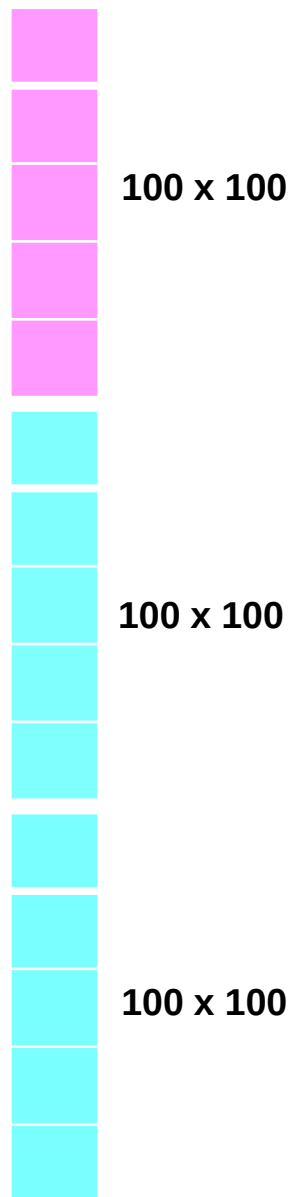
Image Classification



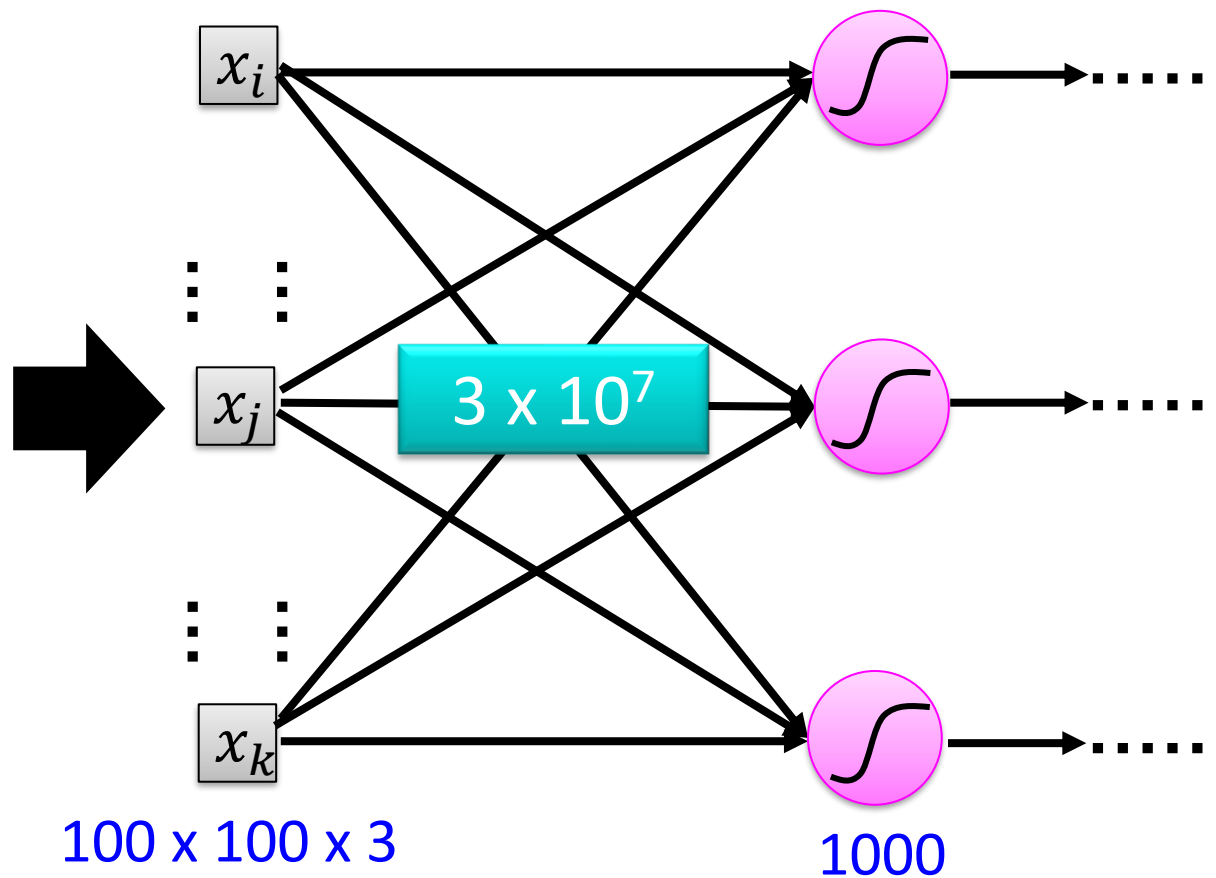
(All the images to be classified have the same size.)

Image Classification





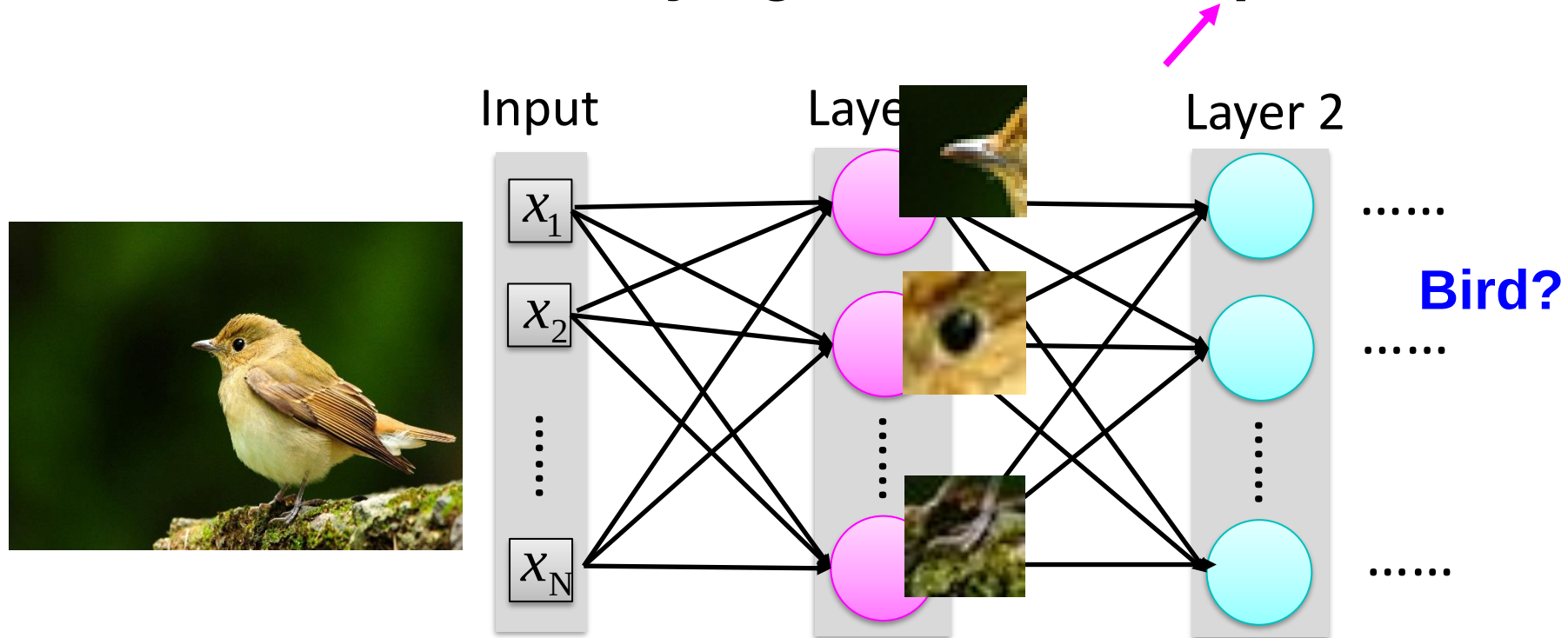
Fully Connected Network



Do we **really** need “fully connected”
in image processing?

Observation 1

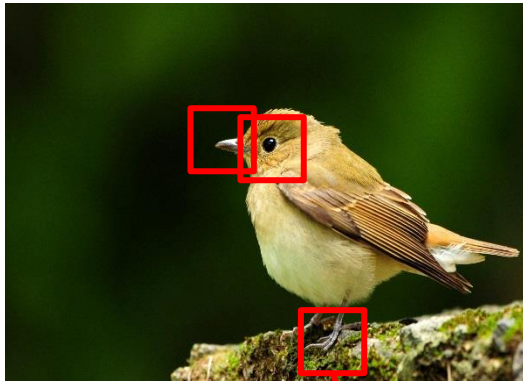
Identifying some critical patterns



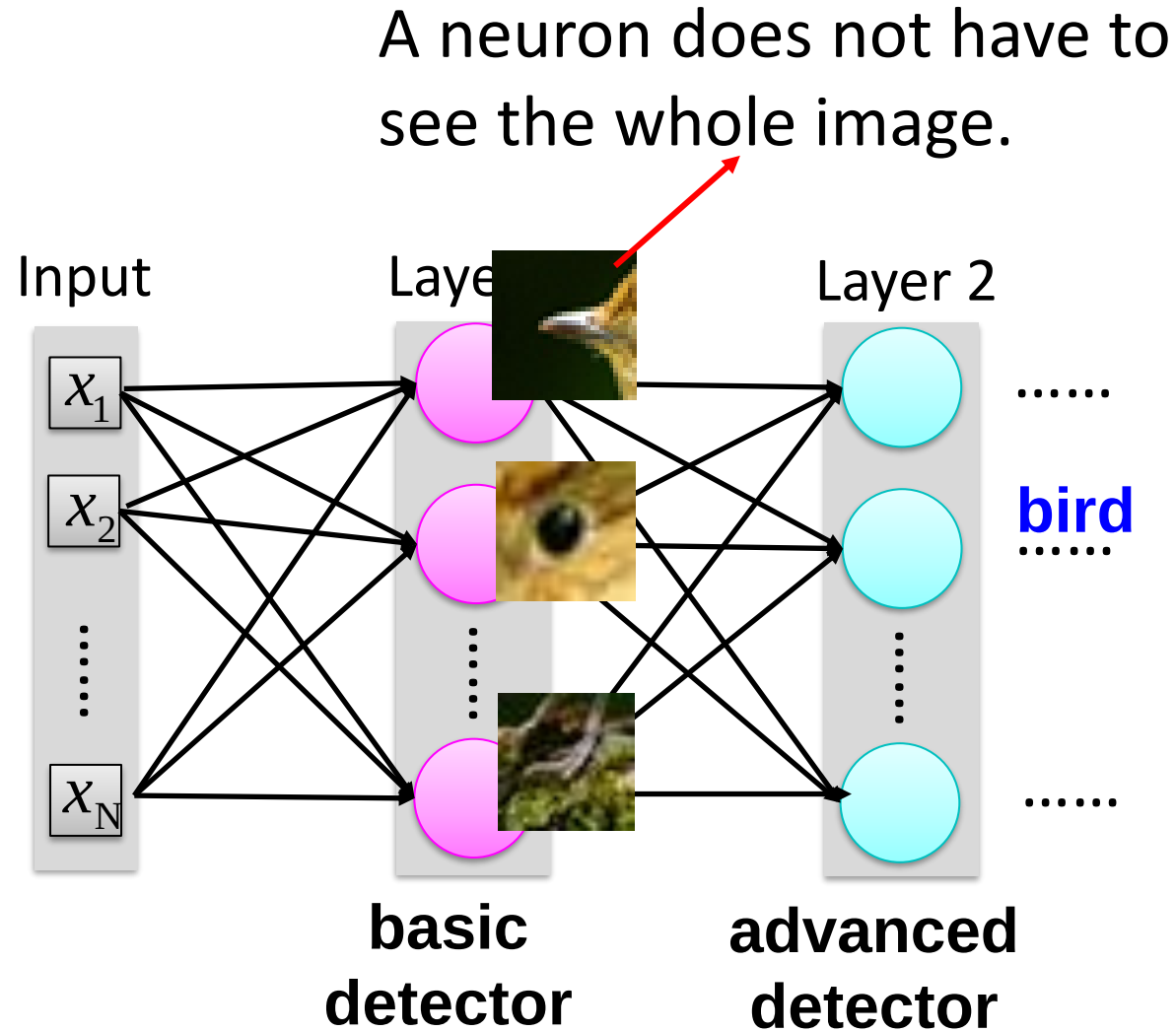
Perhaps human also identify birds in a similar way ... 😊

Observation 1

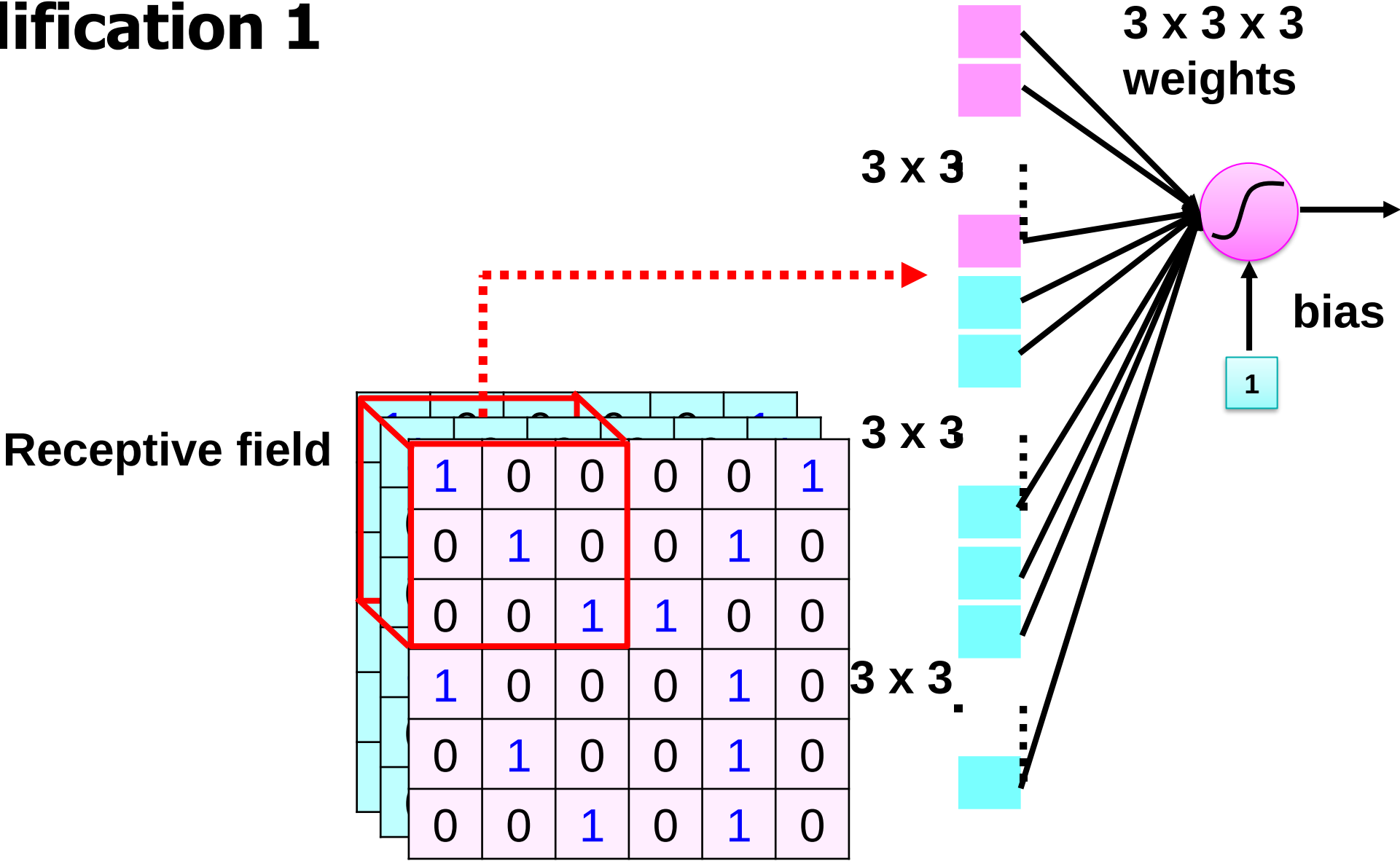
Need to see the whole image?



Some patterns are much smaller than the whole image.

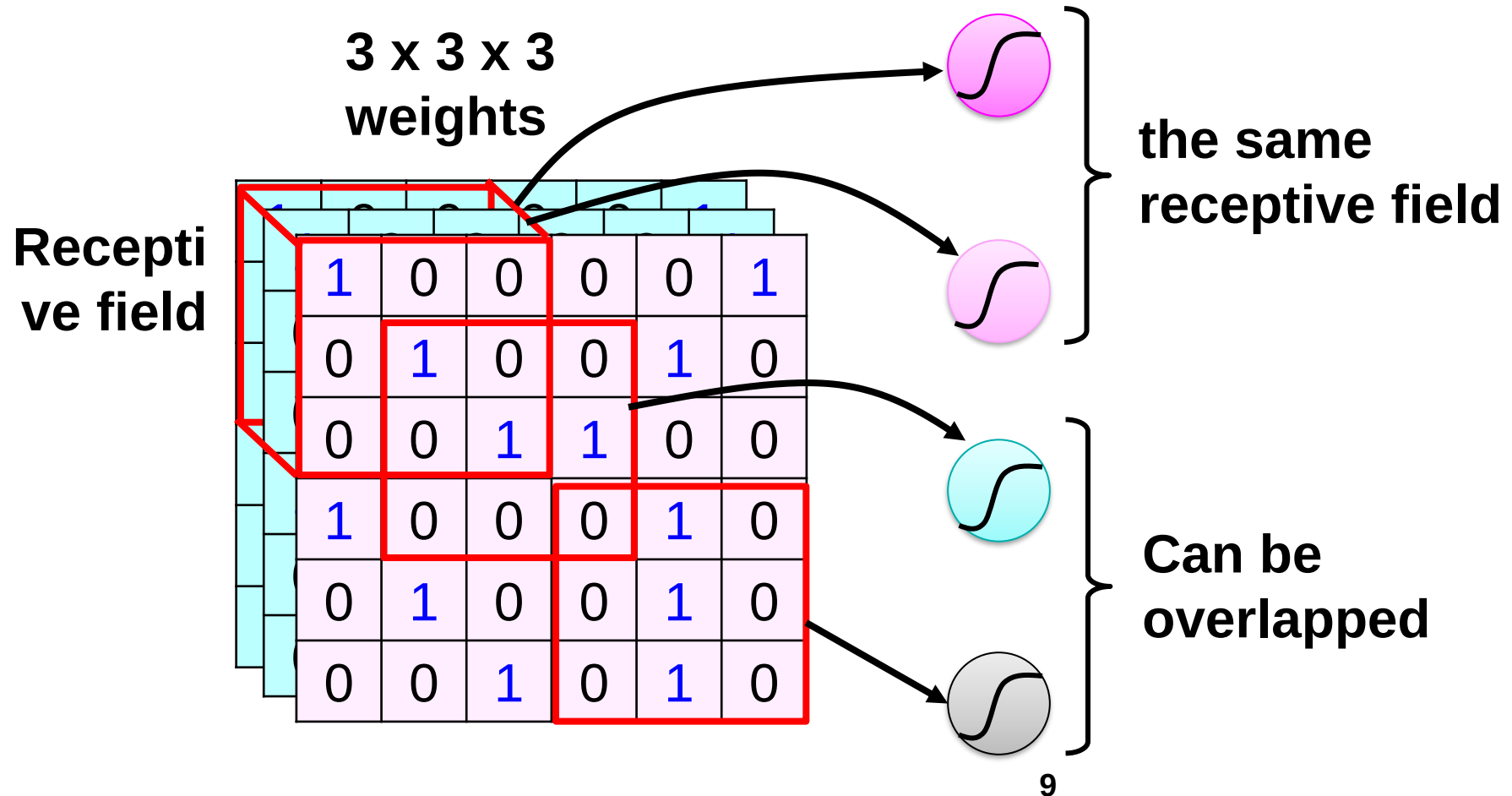


Simplification 1



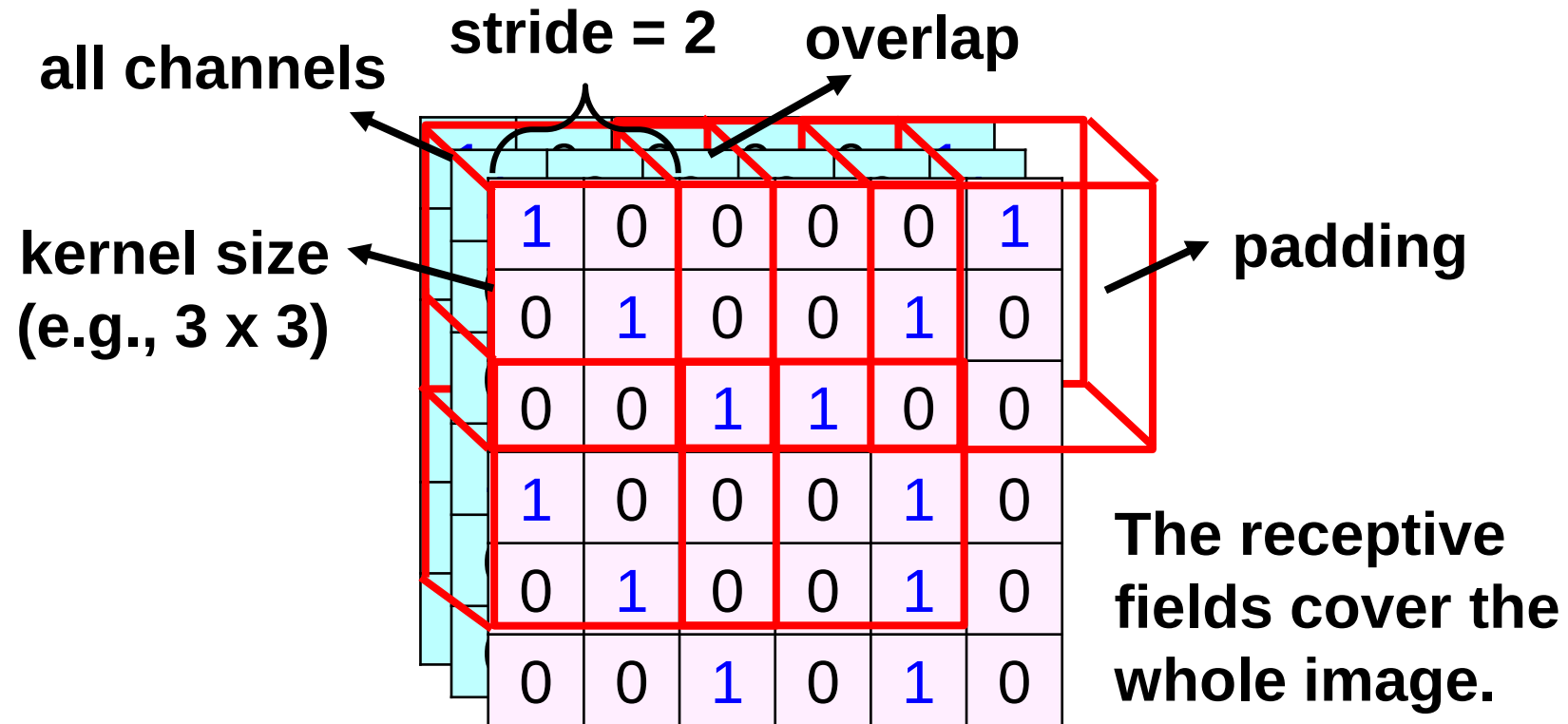
Simplification 1

- Can different neurons have different sizes of receptive field?
- Not square receptive field?



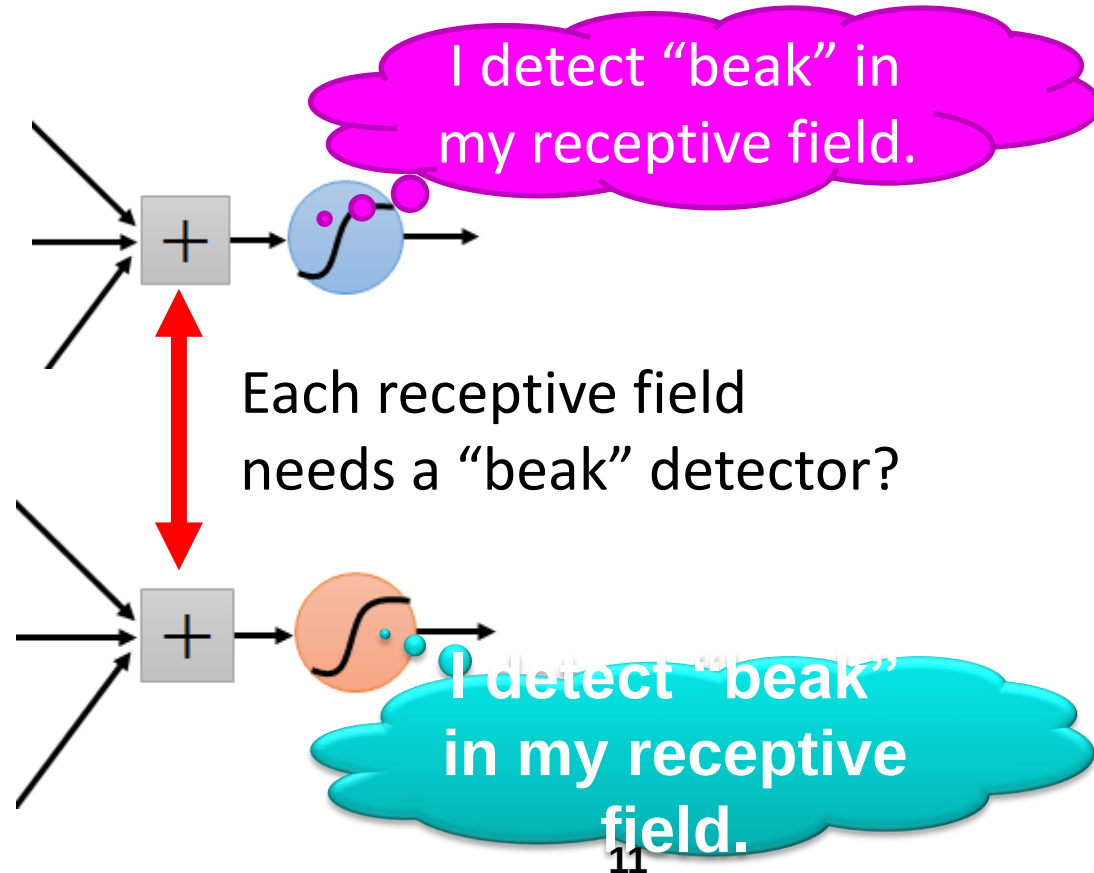
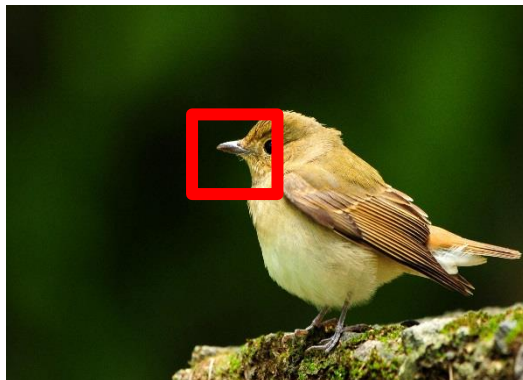
Simplification 1 – Typical Setting

Each receptive field has a set of neurons (e.g., 64 neurons).

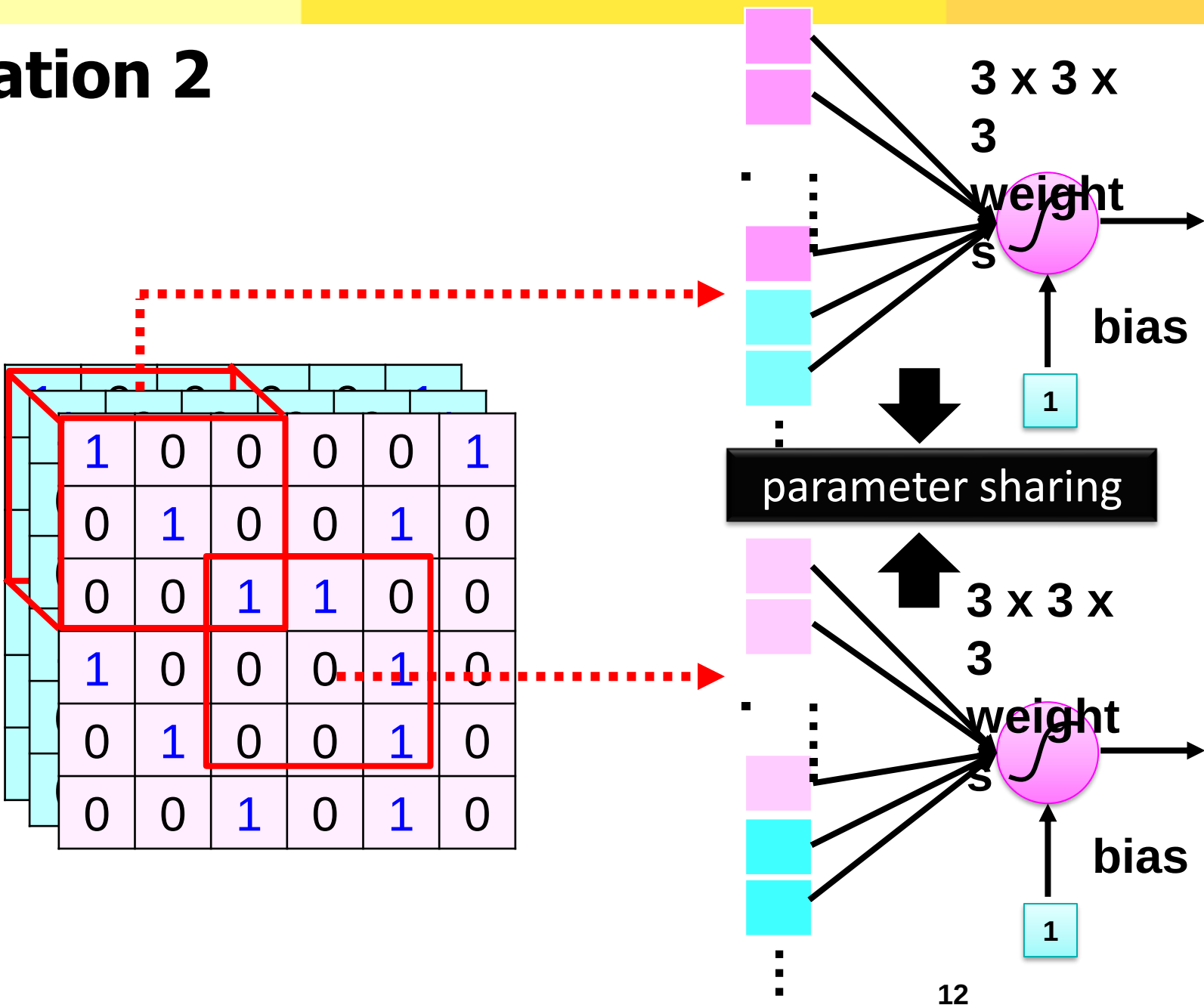


Observation 2

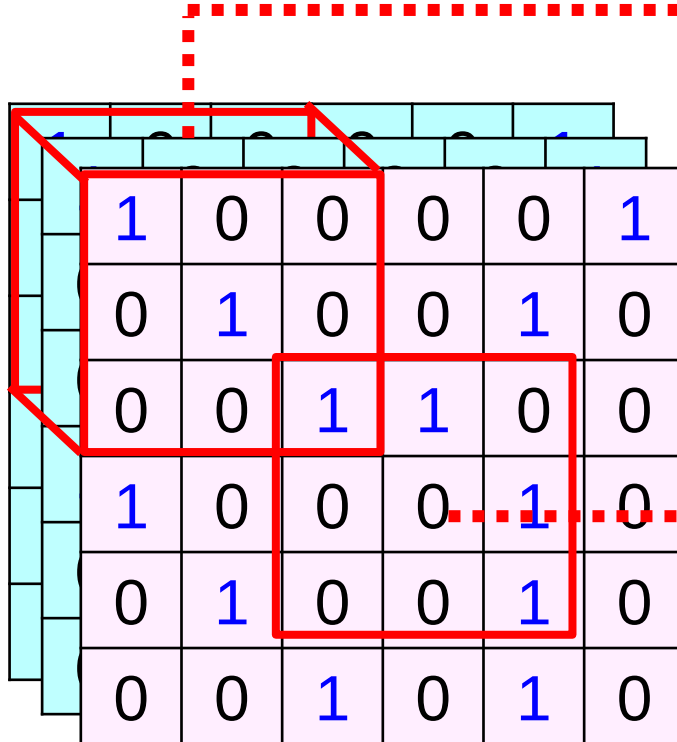
The same patterns appear in different regions.



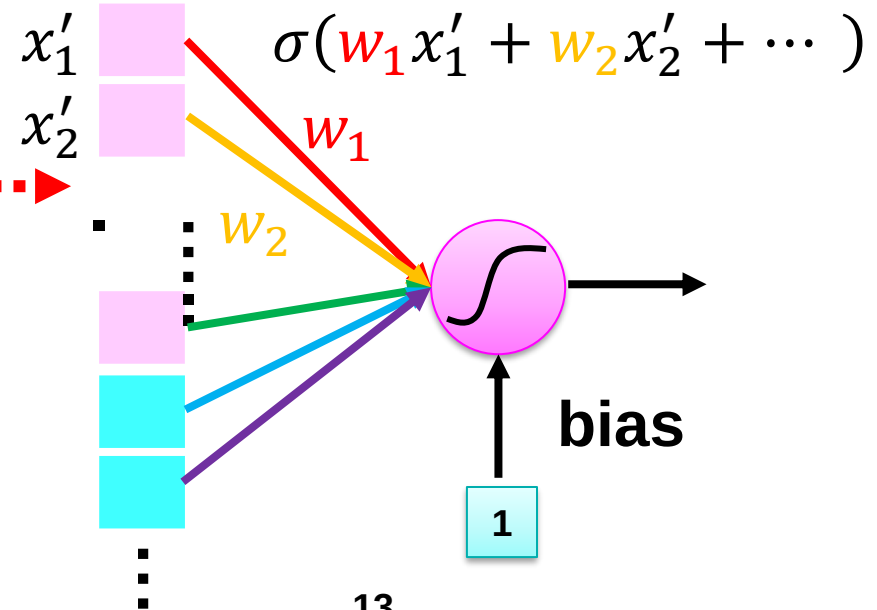
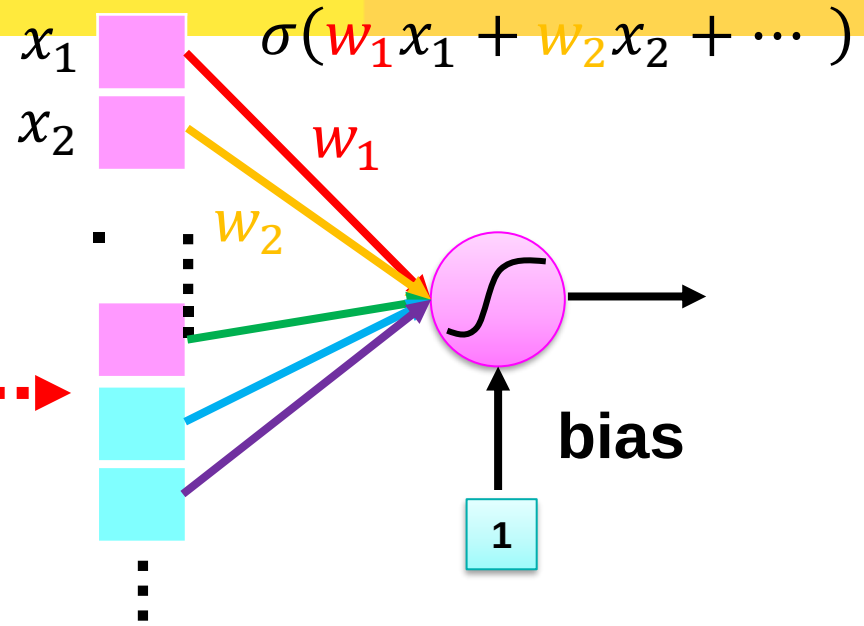
Simplification 2



Simplification 2

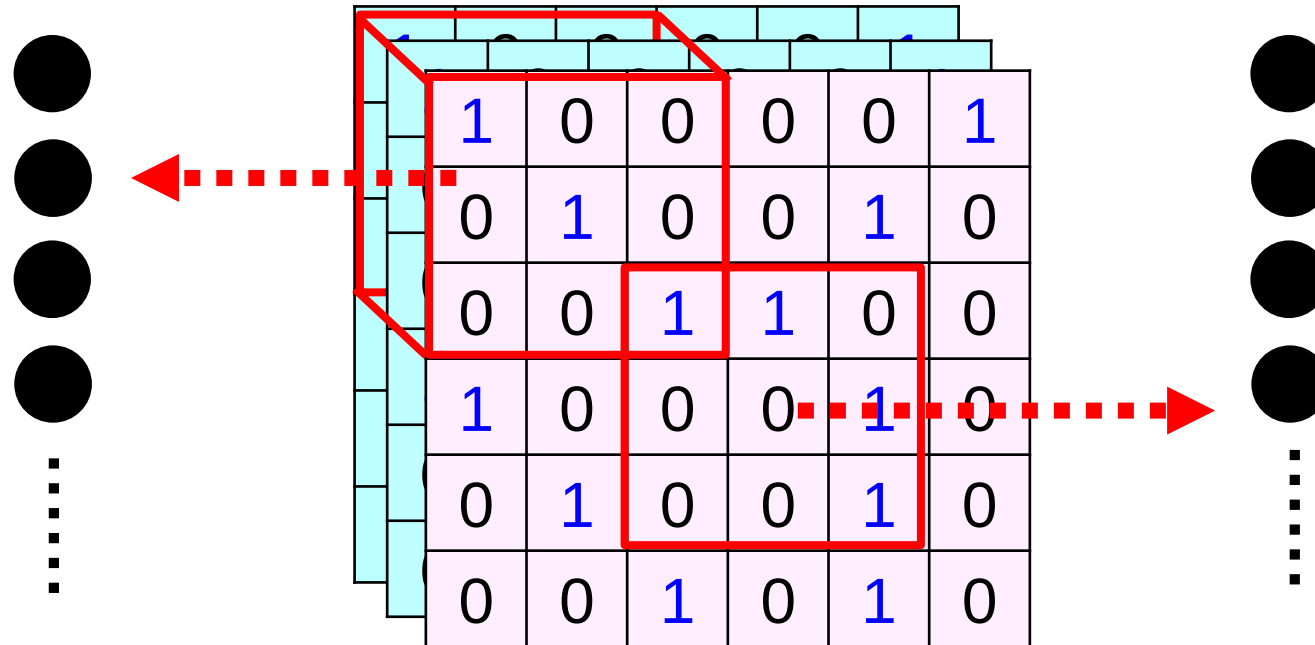


Two neurons with the same receptive field would not share parameters.



Simplification 2 – Typical Setting

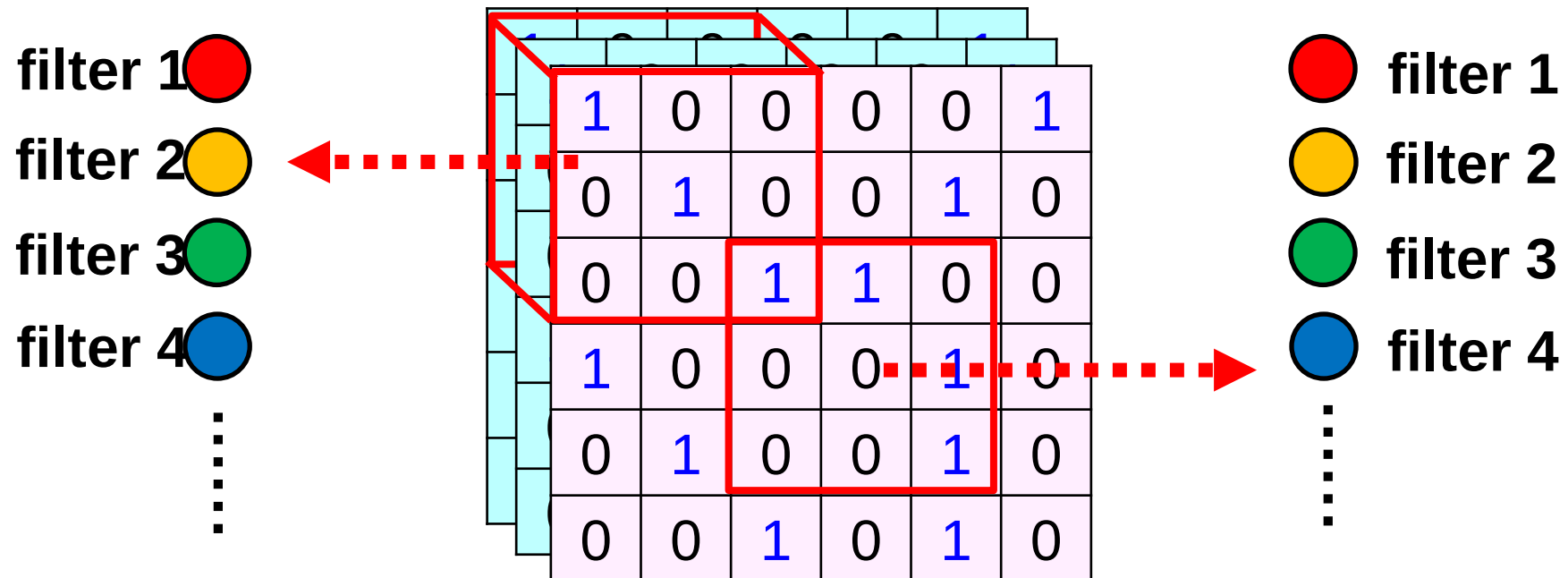
Each receptive field has a set of neurons (e.g., 64 neurons).



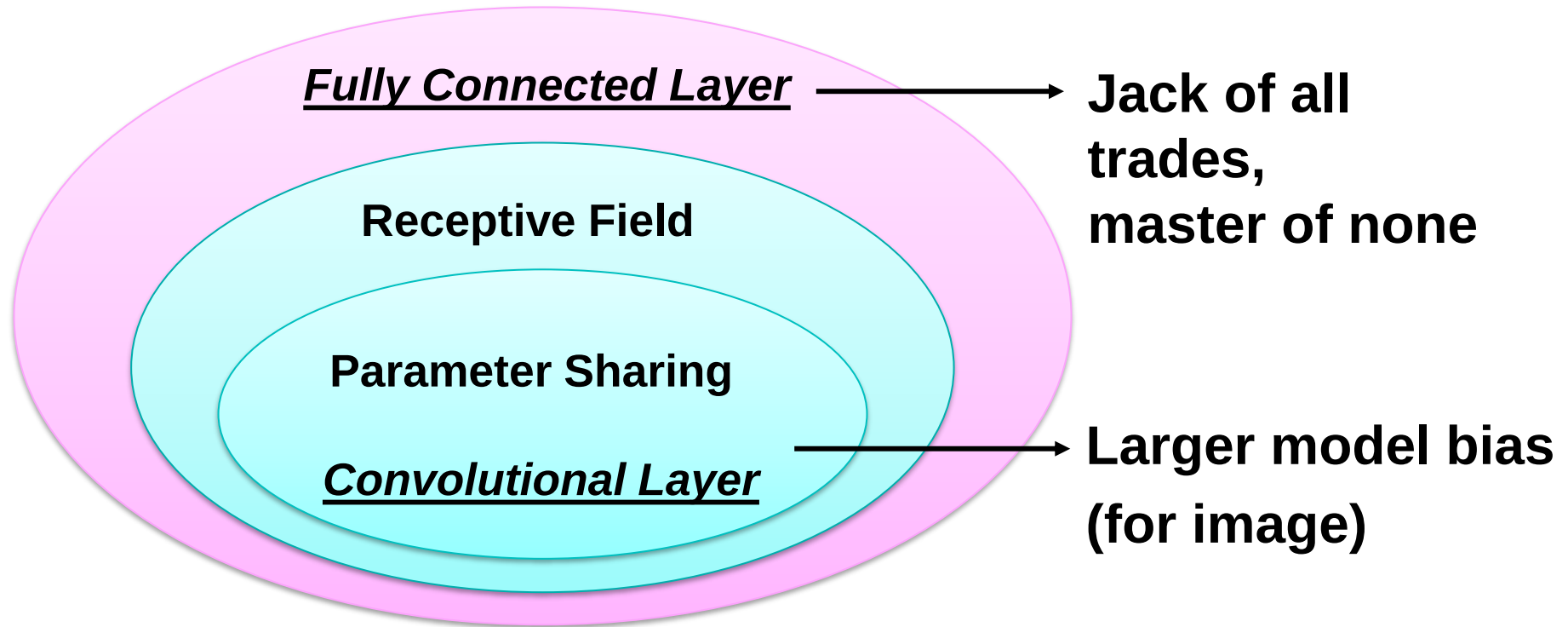
Simplification 2 – Typical Setting

Each receptive field has a set of neurons (e.g., 64 neurons).

Each receptive field has the neurons with the same set of parameters.



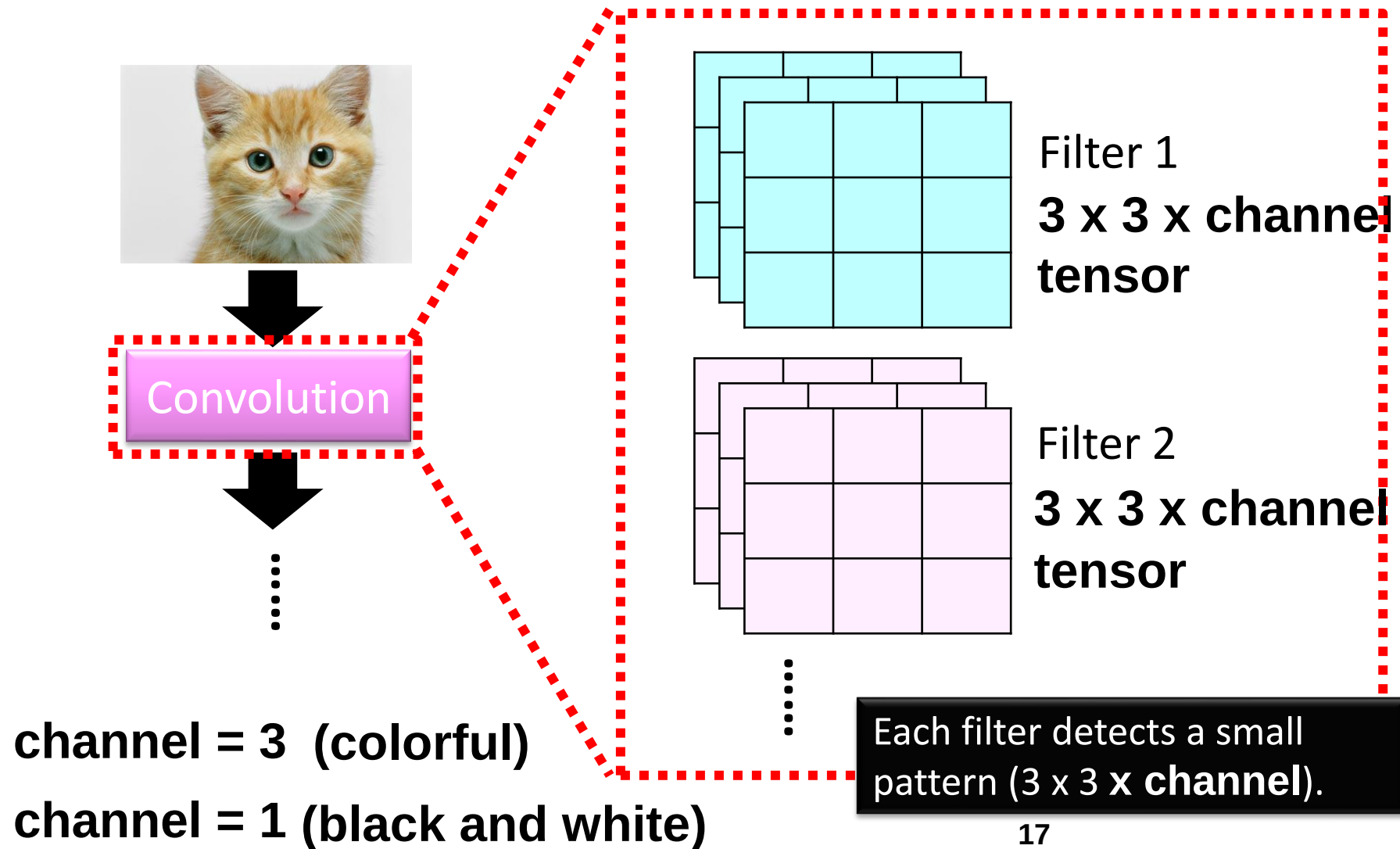
Benefit of Convolutional Layer



- Some patterns are much smaller than the whole image.
- The same patterns appear in different regions.

Convolutional Layer

Another story based on *filter*



Convolutional Layer

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Consider channel = 1
(black and white image)

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮

(The values in the filters
are unknown parameters.)

Convolutional Layer

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Convolutional Layer

stride=1

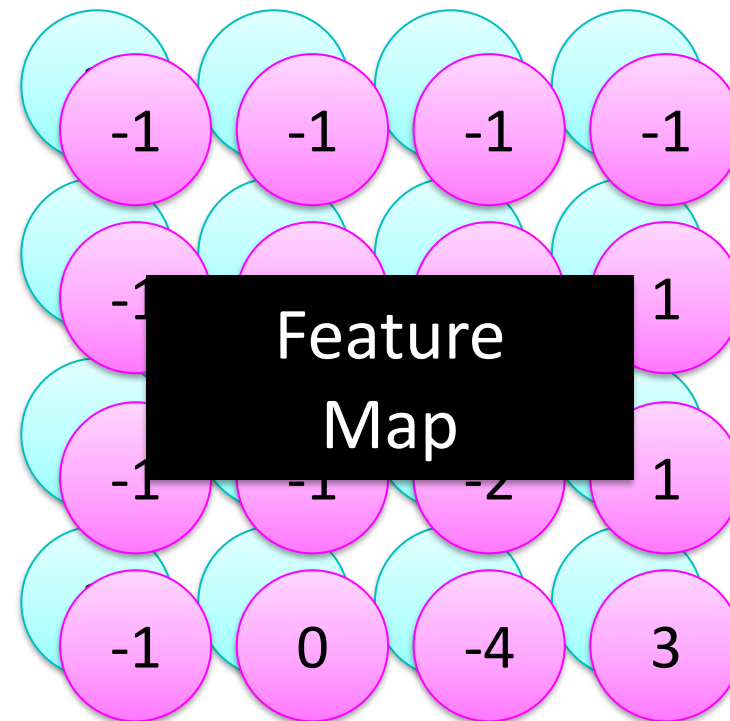
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

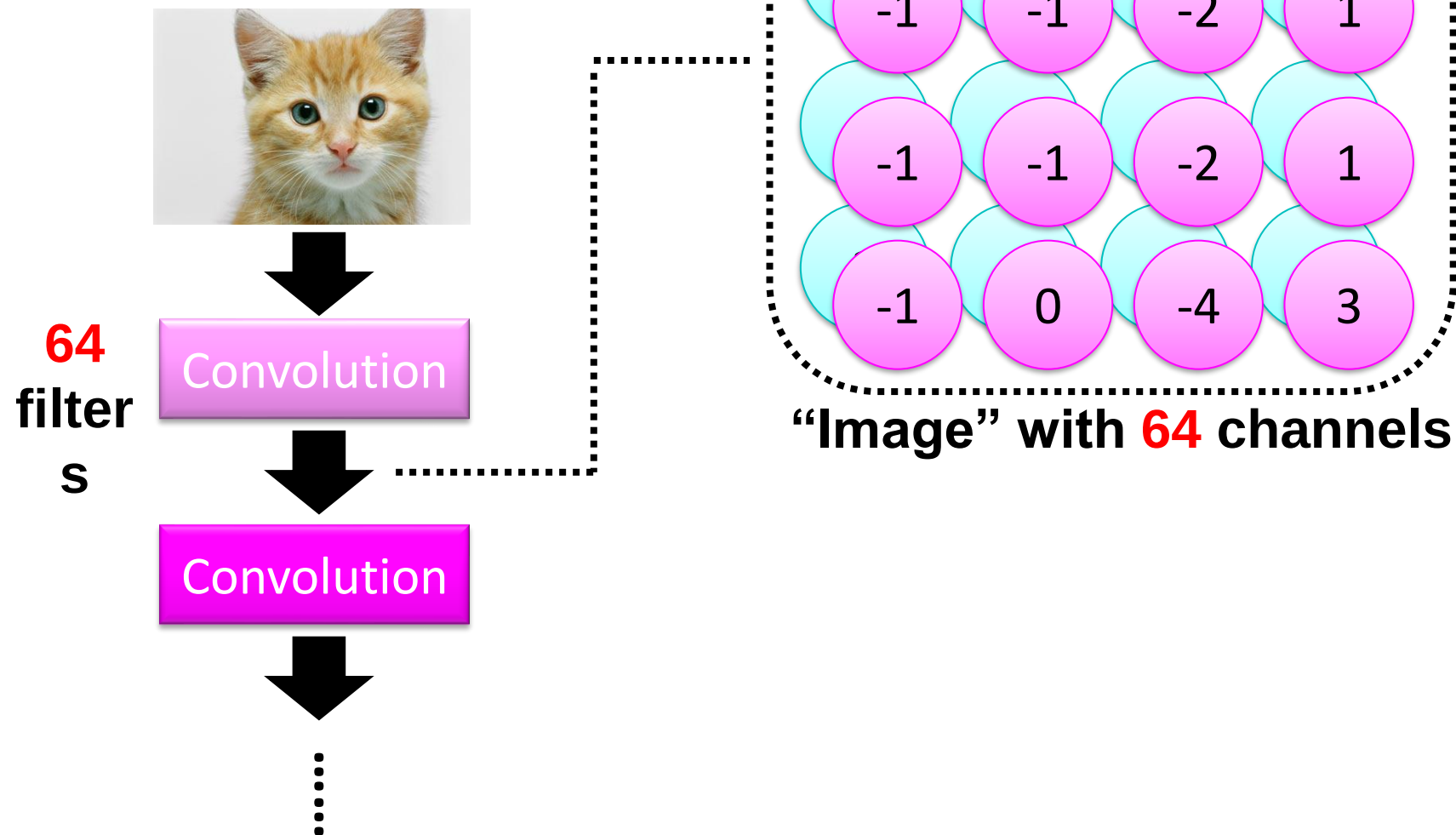
-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

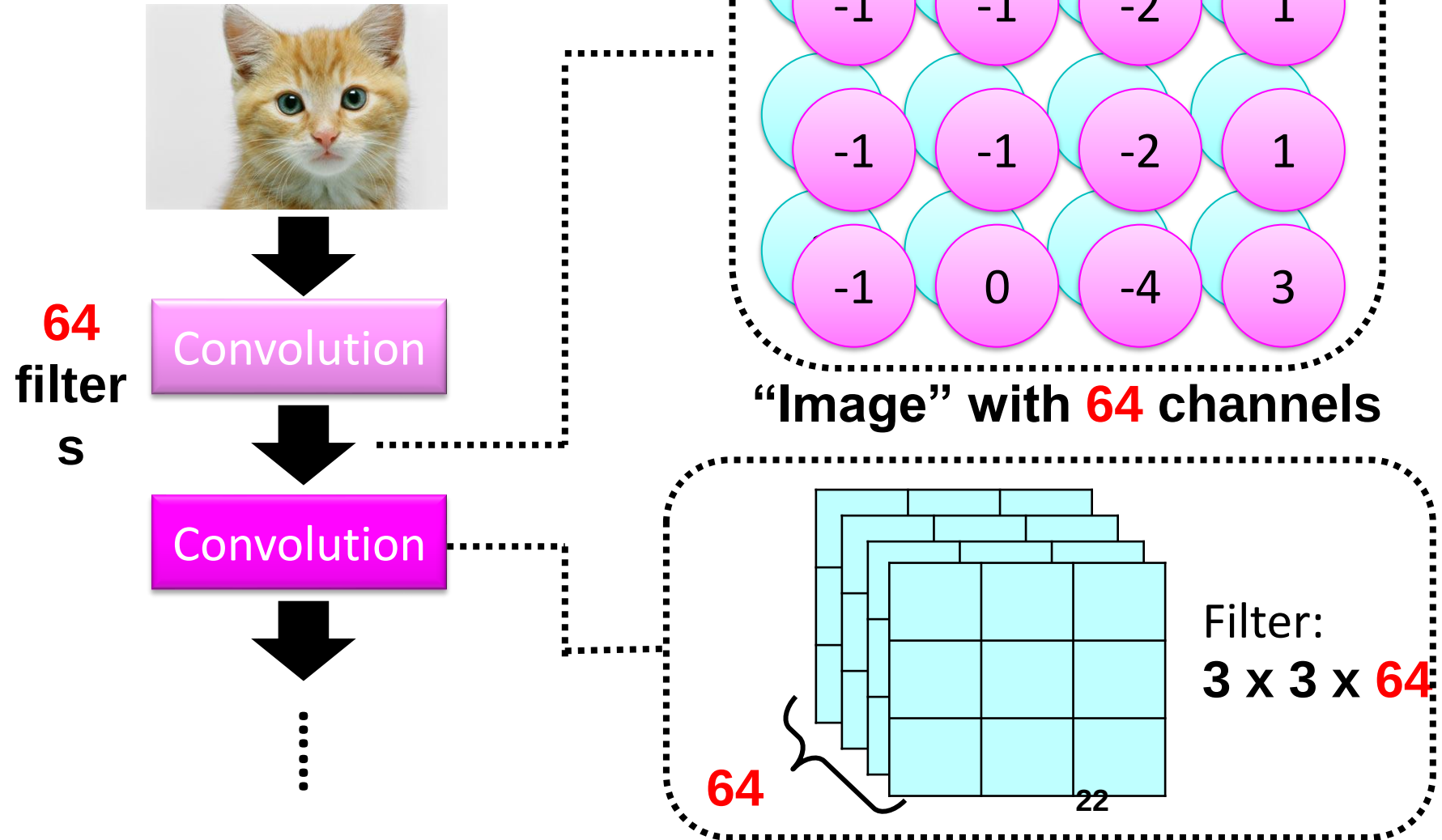
Do the same process for every filter



Convolutional Layer



Multiple Convolutional Layers



Multiple Convolutional Layers



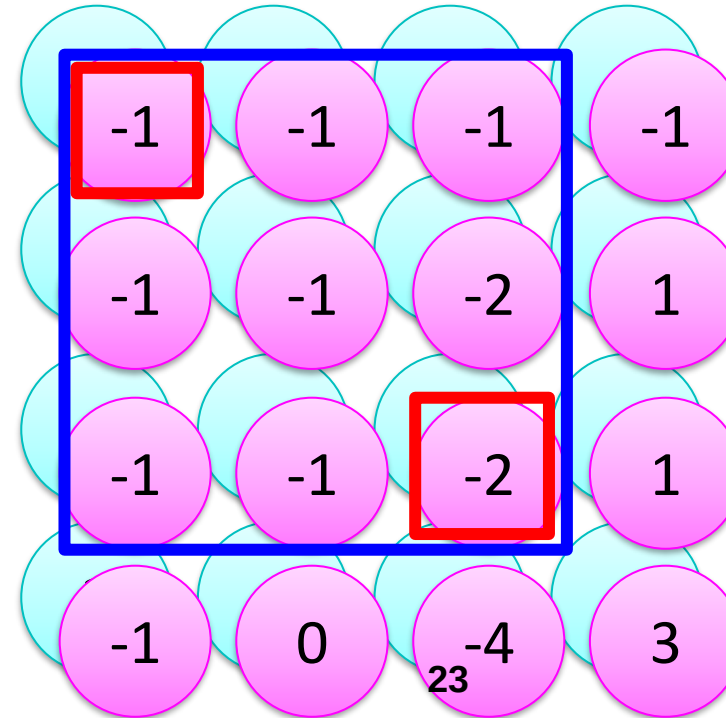
64
filter
s

Convolution

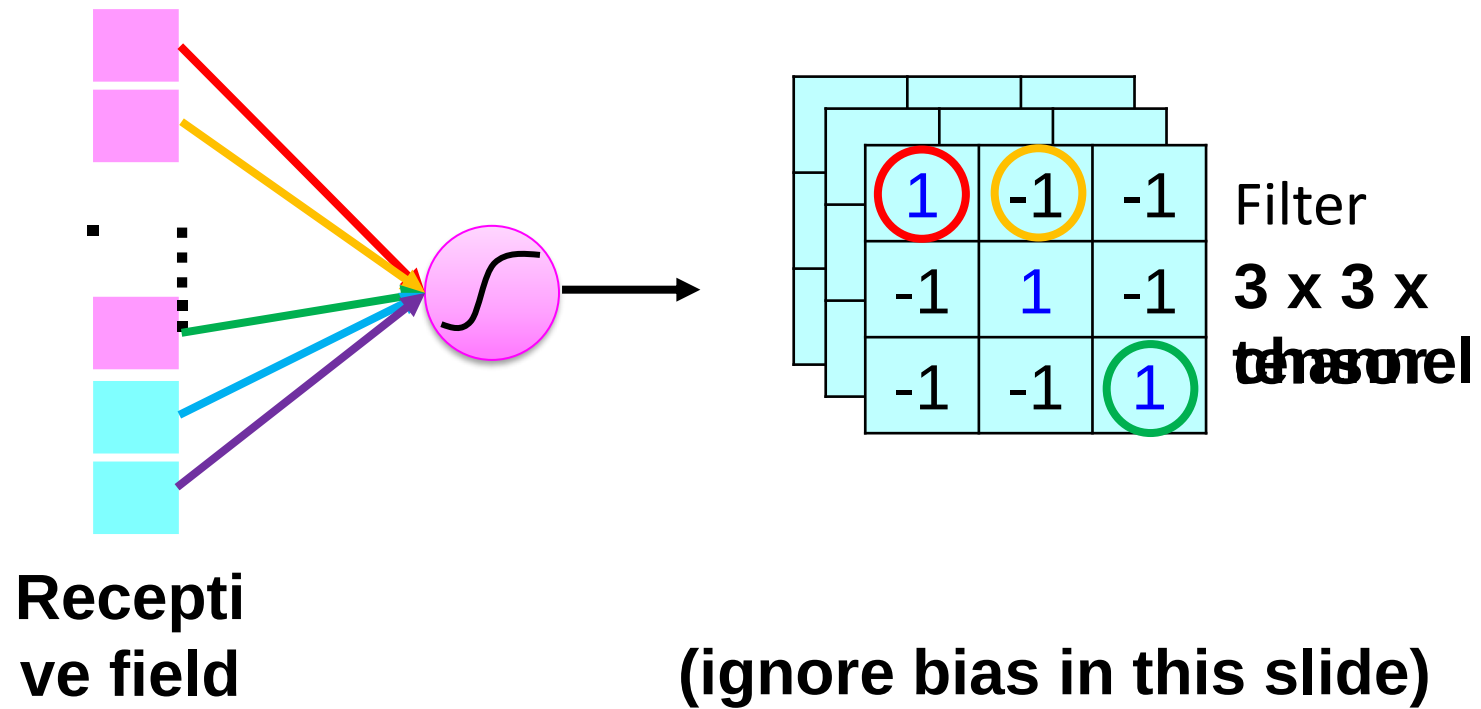
Convolution

⋮

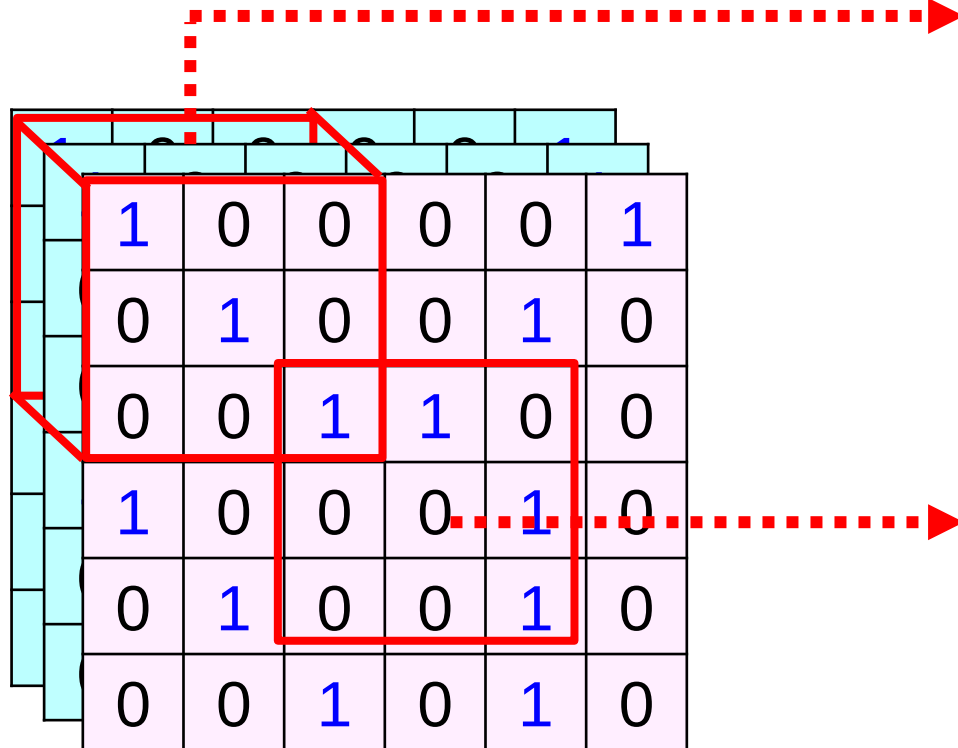
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0



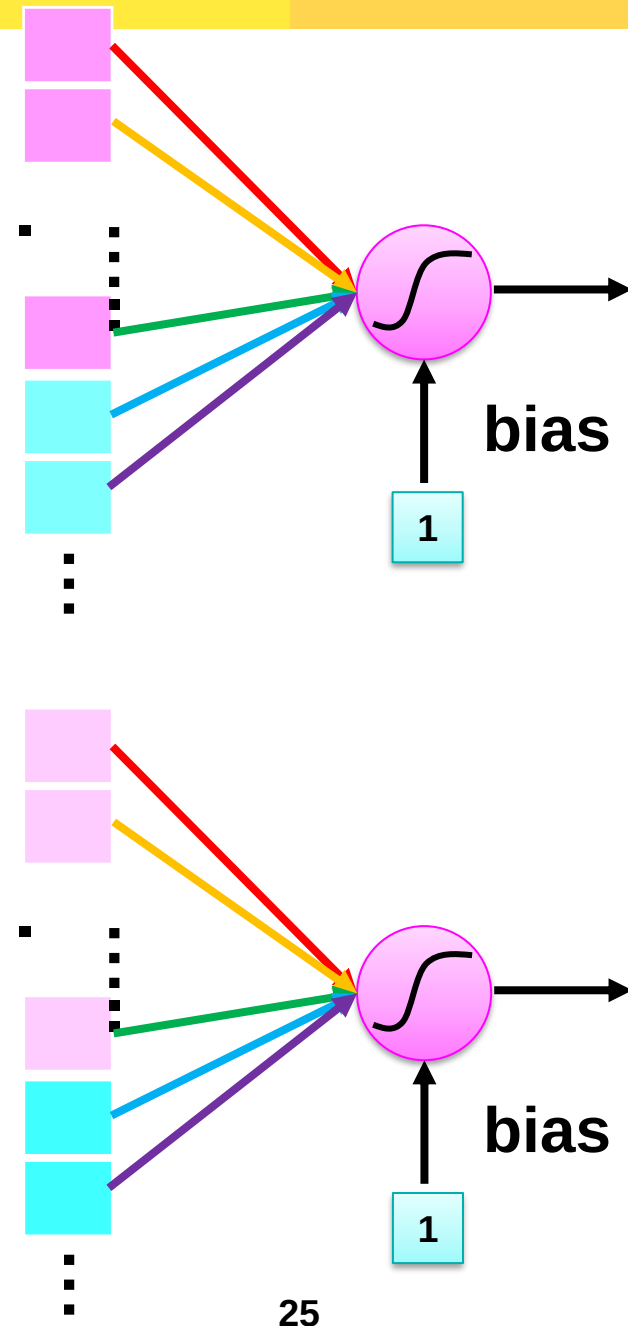
Comparison of Two Stories



The neurons with different receptive fields share the parameters.



Each filter convolves over the input image.



Convolutional Layer

<u><i>Neuron Version Story</i></u>	<u><i>Filter Version Story</i></u>
Each neuron only considers a receptive field.	There are a set of filters detecting small patterns.
The neurons with different receptive fields share the parameters.	Each filter convolves over the input image.

They are the same story.

Observation 3

Subsampling the pixels will not change the object

bird



subsampling

bird



Pooling – Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

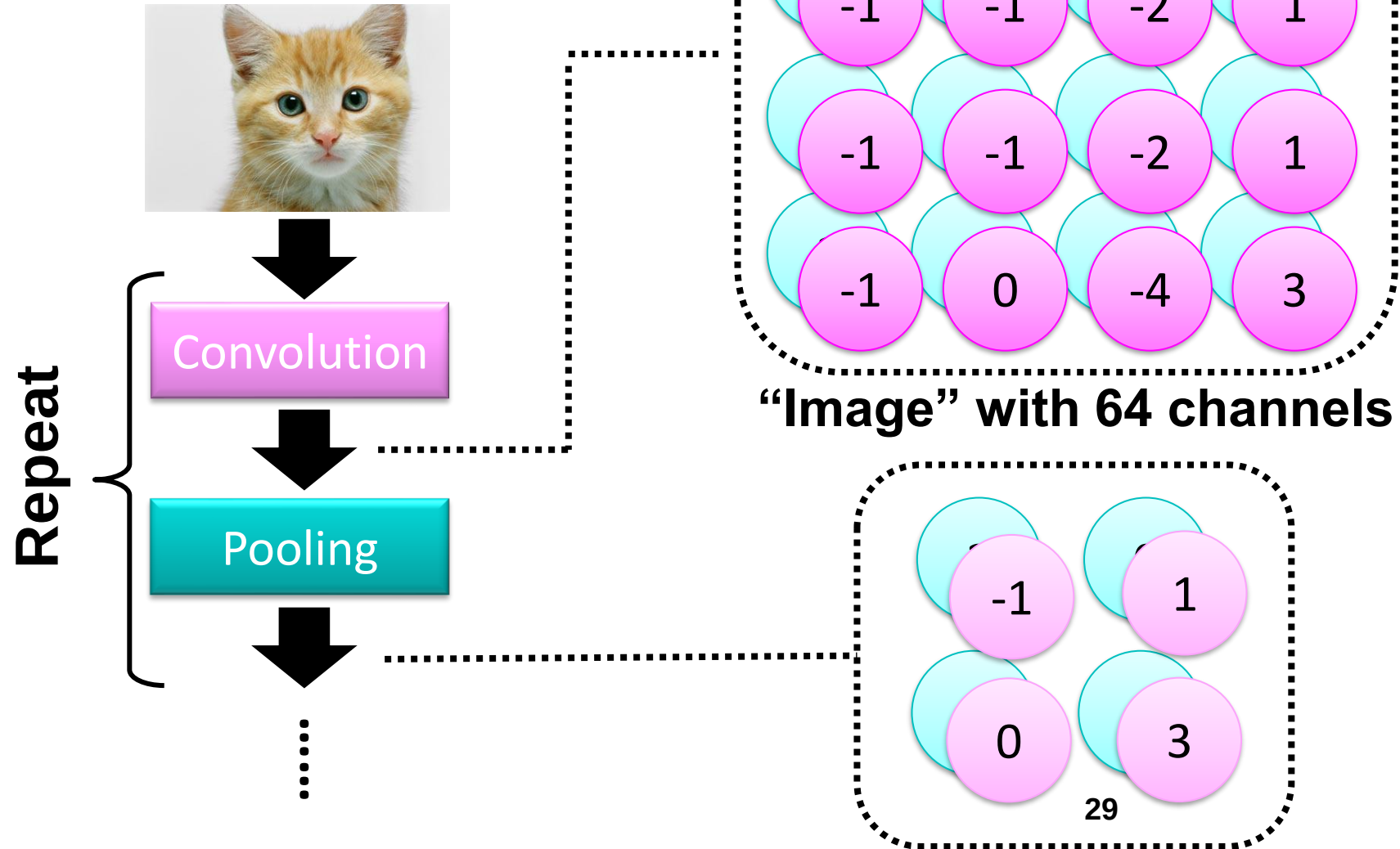
-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

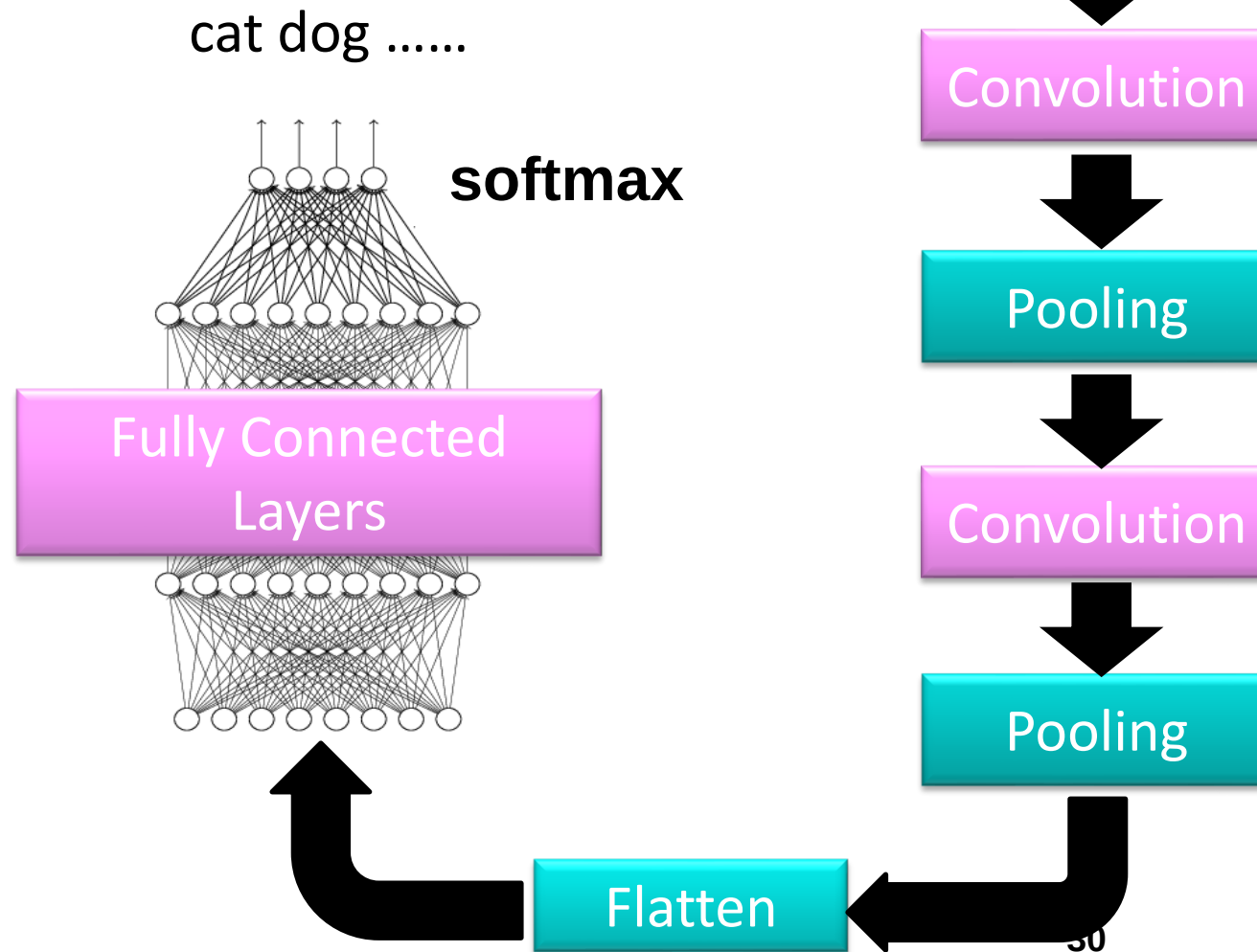
3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

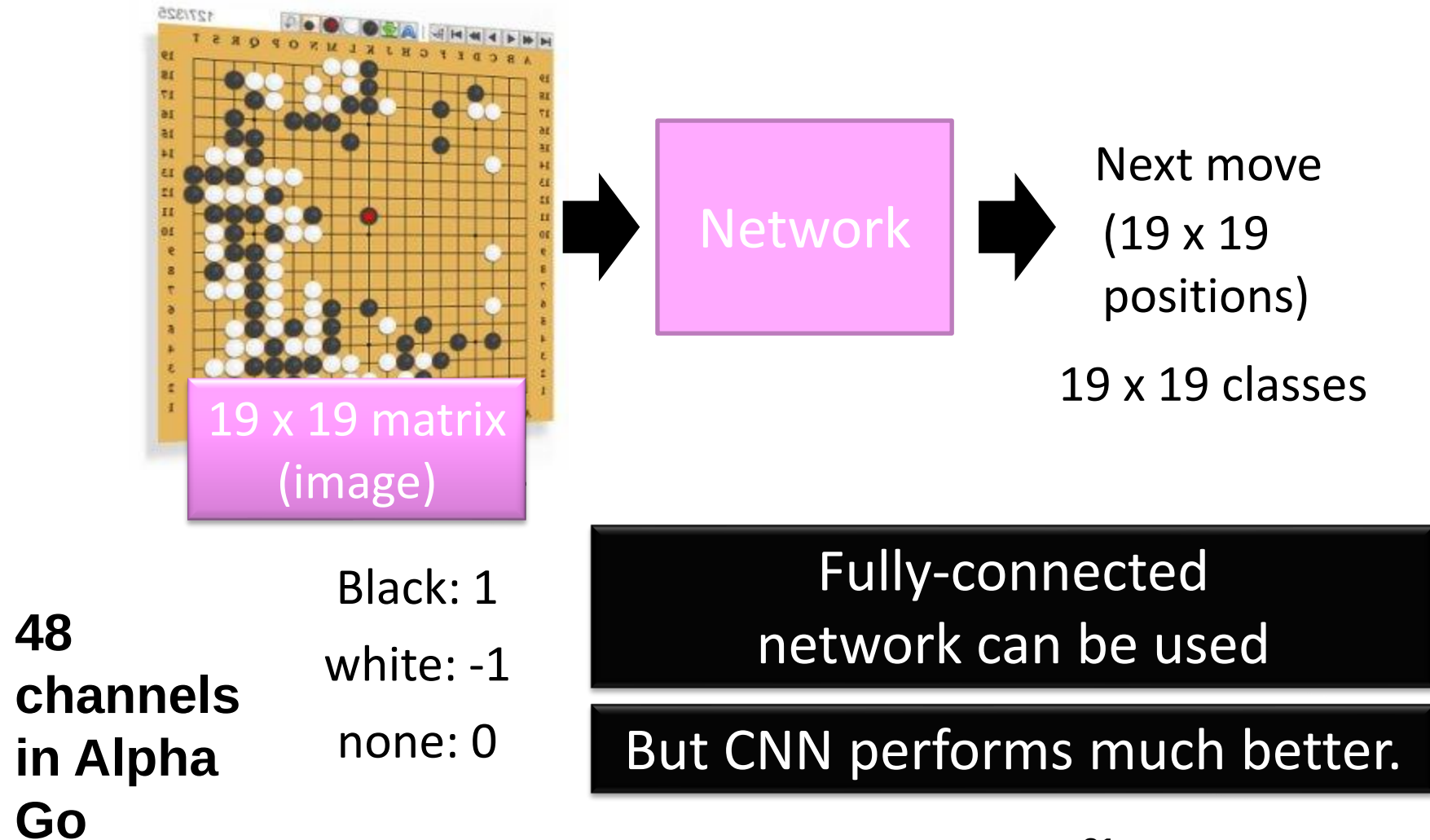
Convolutional Layers + Pooling



The whole CNN



Application: Playing Go

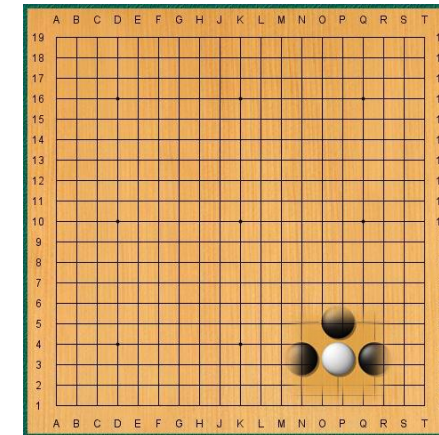
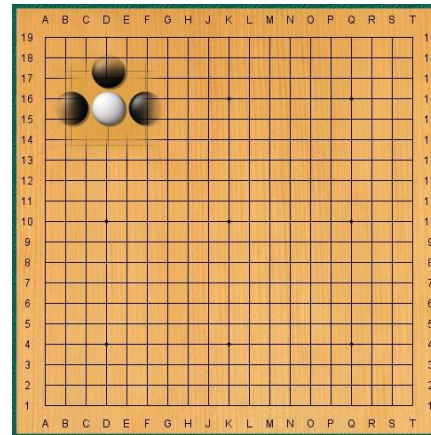
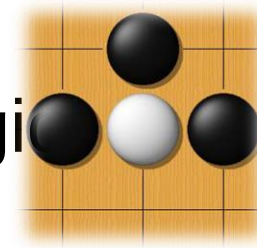


Why CNN for Go playing?

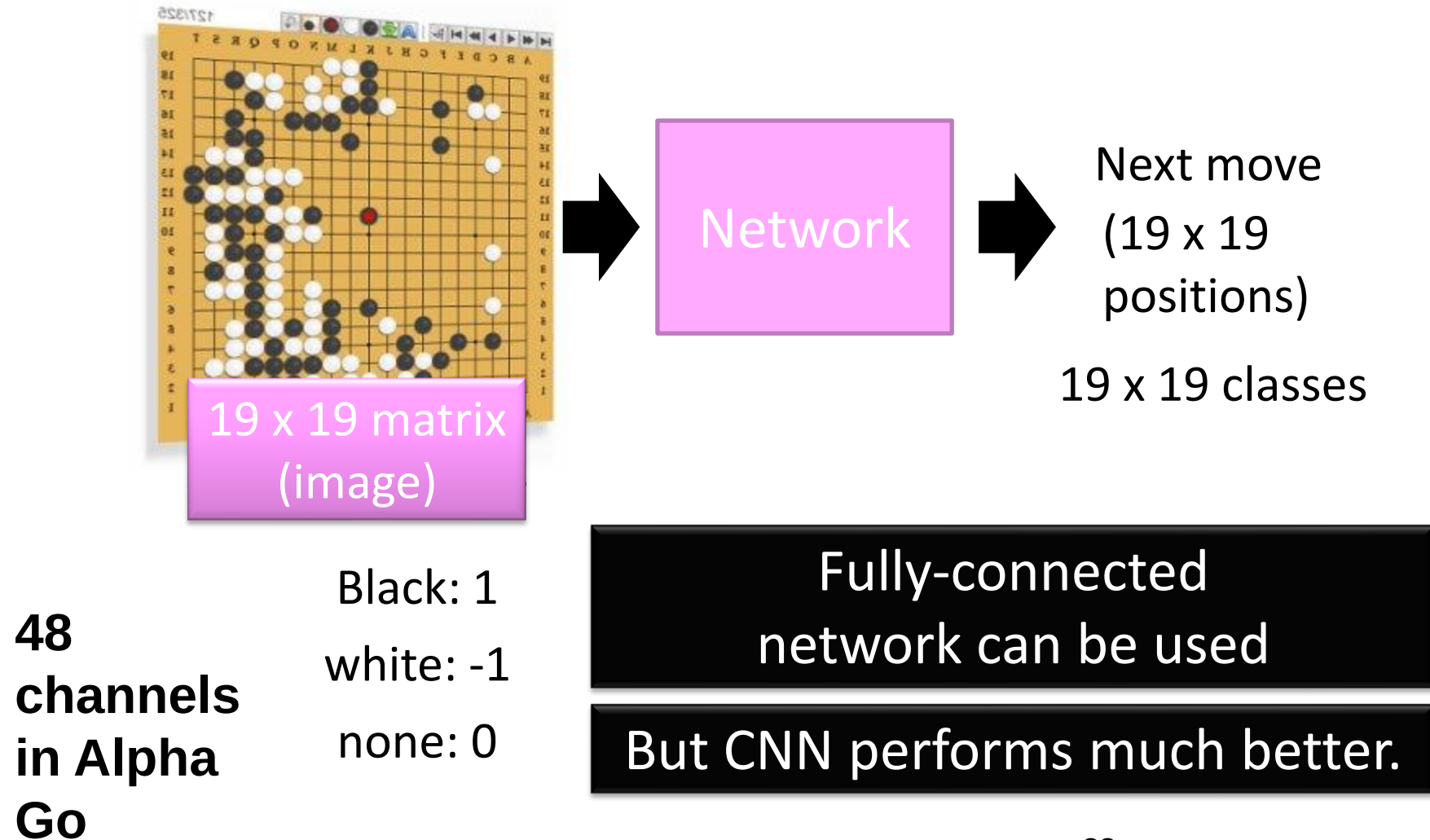
Some patterns are much smaller than the whole image

The same patterns appear in different regions

Alpha Go uses 5 x 5 for first layer

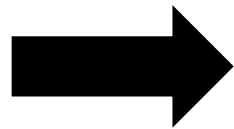


Application: Playing Go



Why CNN for Go playing?

Subsampling the pixels will not change the object



Pooling

How to explain this???

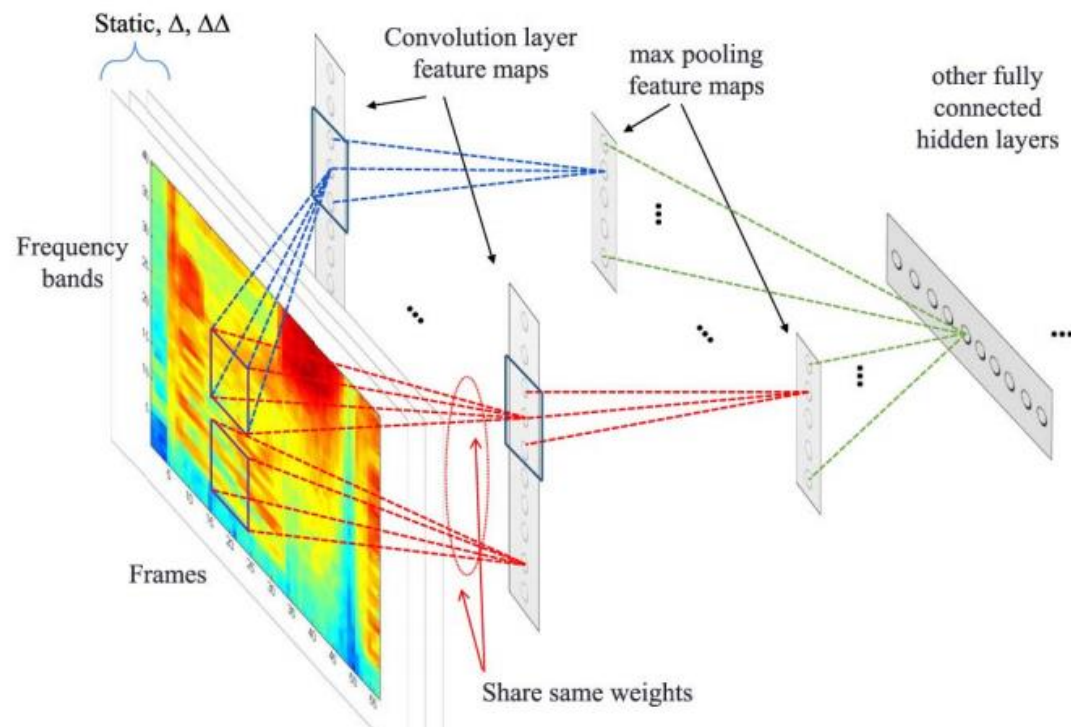
Neural network architecture. The input to the policy network is a $19 \times 19 \times 48$ image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23×23 image, then convolves k filters of kernel size 5×5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21×21 image, then convolves k filters of kernel size 3×3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1×1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used $k = 192$ filters; Fig. 2b and Extended Data Table 1. The extended version used 256 and 384 filters.

Alpha Go does not use Pooling

More Applications

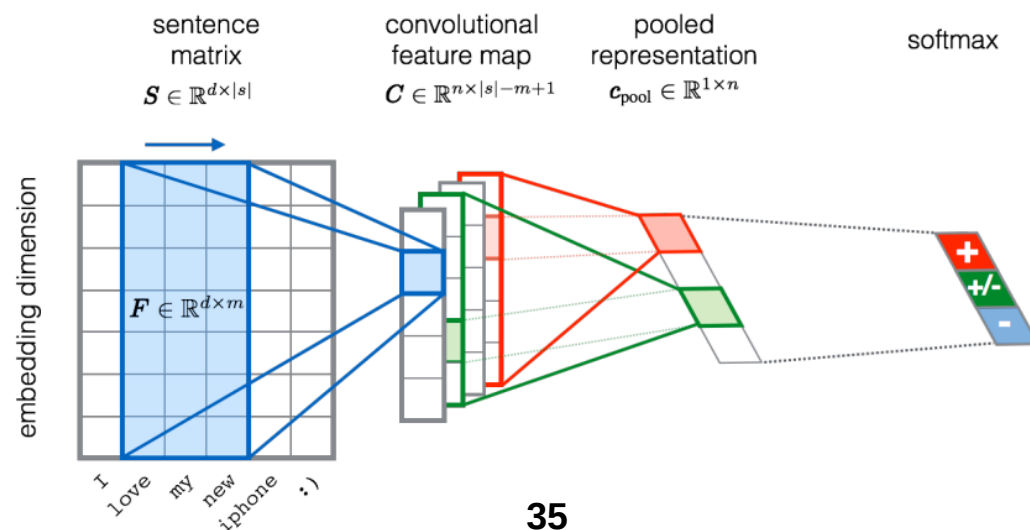
Speech

<https://dl.acm.org/doi/10.1109/TA.SLP.2014.2339736>



Natural Language Processing

<https://www.aclweb.org/anthology/S15-2079/>



To learn more ...

CNN is not invariant to scaling and rotation (we need data augmentation).